```
import numpy as np
import pandas as pd
from ISLP import load_data

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
```

```
College = load_data("College")
College = College.dropna()

College = pd.get_dummies(College, columns=["Private"], drop_first=True)

# Response: Apps
y = College["Apps"].values

# Predictors: 所有非 Apps 欄位
X = College.drop(columns=["Apps"]).values

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)
```

```
Train size: (543, 17)
Test size : (234, 17)
```

```
ols = make_pipeline(StandardScaler(), LinearRegression())
ols.fit(X_train, y_train)
y_pred_ols = ols.predict(X_test)
mse_ols = mean_squared_error(y_test, y_pred_ols)

print("\n(b) OLS Test MSE:", mse_ols)
```

```
(b) OLS Test MSE: 642753.8976533797
```

```
lambdas = np.logspace(-2, 5, 200)
ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=lambdas, cv=10))
ridge.fit(X_train, y_train)
```

```
y_pred_ridge = ridge.predict(X_test)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)

best_lambda_ridge = ridge.named_steps['ridgecv'].alpha_

print("\n(c) Ridge Test MSE:", mse_ridge)
print("    Best lambda:", best_lambda_ridge)
```

```
(c) Ridge Test MSE: 679292.1143411023
    Best lambda: 10.59560179277616
```

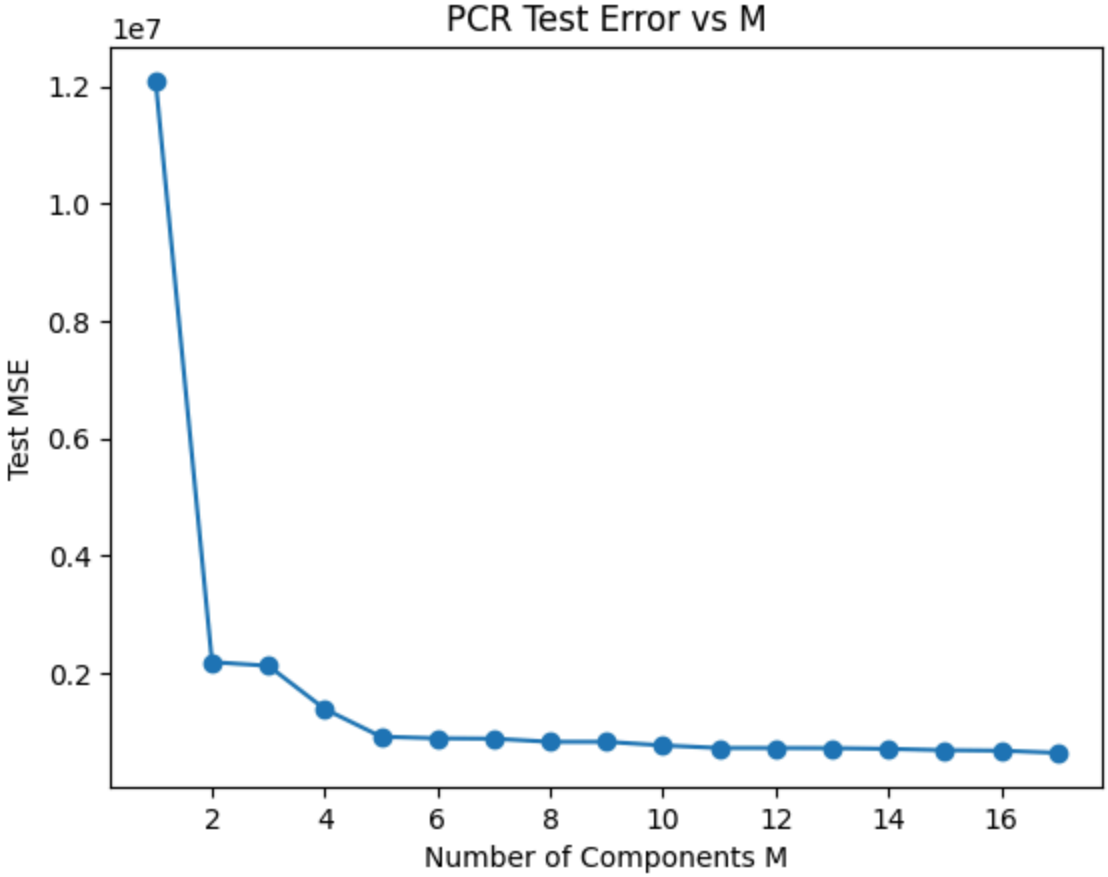
```
lasso = make_pipeline(
    StandardScaler(),
    LassoCV(cv=10, random_state=1)
)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)

lasso_model = lasso.named_steps["lassocv"]
coef = lasso_model.coef_
nonzero = np.sum(coef != 0)

print("\n(d) Lasso Test MSE:", mse_lasso)
print("    Best lambda:", lasso_model.alpha_)
print("    Non-zero coefficients:", nonzero)
```

```
(d) Lasso Test MSE: 659813.5889970256
    Best lambda: 12.377876212324587
    Non-zero coefficients: 14
```

```
def pcr_cv(X_train, y_train, X_test, y_test, max_M=20):
    n_features = X_train.shape[1]
    max_M = min(max_M, n_features)

    mse_cv = []

    for M in range(1, max_M+1):
        pca = PCA(n_components=M)
        Xtr_pca = pca.fit_transform(X_train)

        model = LinearRegression()
        model.fit(Xtr_pca, y_train)

        Xte_pca = pca.transform(X_test)
        pred = model.predict(Xte_pca)
```

```
        mse = mean_squared_error(y_test, pred)
        mse_cv.append(mse)

    best_M = np.argmin(mse_cv) + 1
    best_mse = mse_cv[best_M-1]

    return best_M, best_mse, mse_cv

best_M, mse_pcr, mse_list = pcr_cv(X_train, y_train, X_test, y_test, max_M=20)

print("\n(e) PCR Test MSE:", mse_pcr)
print("    Best M:", best_M)

plt.plot(range(1, len(mse_list)+1), mse_list, marker='o')
plt.xlabel("Number of Components M")
plt.ylabel("Test MSE")
plt.title("PCR Test Error vs M")
plt.show()
```

```
PCR Test MSE: 642753.8976533444
    Best M: 17
```


PCR Test Error vs M
1e7
1.2
1.0
0.8
0.6
0.4
0.2
Test MSE
2
4
6
8
10
12
14
16
Number of Components M